

refactor-2 — Trade & SOR Architecture

Branch: refactor-2 · **Repos:** prinx-interface + predictions-api · May 2026

This document describes the *refactor* (module boundaries, unified quote/execute path, VACM, corridor prefund) — not a generic product overview.

1. Refactor goals

- **Fewer redundant calls** — one SOR route poll; corridor-scoped RPC on execute (not all chains every time).
- **Trade box split** — ~3k-line monolith → controller + hooks (~940 lines in `PredictionMarketTradeBox.tsx`).
- **Unified quoting** — DFlow Pond overlay on `POST /api/sor/route`; deleted `useDflowOrderQuote.ts` and `GET /api/dflow/order/quote`.
- **VACM as single wallet map** — `accountWallets.ts`; prefund reads `chainsForBridgeCorridor()` only.
- **Fail loud on missing wallets** — partial via `walletGate` / `isTradeFundingReady` (not everywhere yet).

2. Module map (what moved where)

```
PredictionMarketTradeBox/
├── PredictionMarketTradeBox.tsx      UI shell (~940 lines)
├── hooks/
│   ├── useTradeBoxController/      Phase A orchestrator
│   │   ├── index.ts                funding → quotes → execute → button
│   │   ├── useTradeBoxSorFunding.ts walletBalances + positions for SOR
│   │   ├── useTradeBoxQuotesLayer.ts derive amount + useSorRoute + useTradeQuote
│   │   ├── useTradeBoxSorExecution.ts useSorExecution + post-trade sync
│   │   ├── useTradeBoxTradeButton.ts useButtonState / venueButtonState
│   │   └── useTradeBoxVenueSelection.ts
│   ├── useTradeBoxVenueWiring.ts    orderbooks, Predict/Poly sessions
│   ├── deriveSorRouteAmount.ts      amount string → SOR numeric input
│   ├── useButtonState.ts / venueButtonState/
│   └── useTradeState.ts
└── tradeQuote/                      display preview (not a second quote API)

src/trading/sor/
├── core/                            useSorRoute, useSorExecution, useSorLegExecutor, sor-types, sor-api
├── route/                           sorPreflight, sorUiUtils, sorQuoteTrust
├── prefund/                          runBridgePrefund, prefundPlan, fundingStableBalances, LiFi quote solve
├── post-trade/                       usePostTradeAccountSync, reconcile, baseline, refresh
└── execute/                          dispatchLeg → venues/*/execute/executeLeg.ts

src/trading/venues/{polymarket,predict,dflow,limitless,levelup}/
└── execute/executeLeg.ts             one file per venue (SOR dispatch target)
```

```

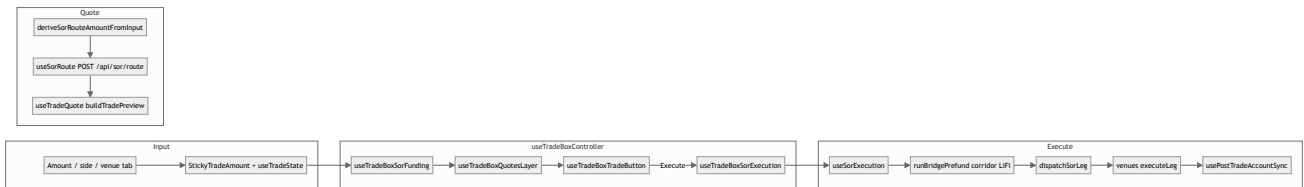
src/context/
├─ AccountDataContext.tsx          session hub (profile, VACM, positions, cash)
├─ accountWallets.ts              normalizeWalletRolesFromOverview → VACM

predictions-api/
├─ api/private-api/sor/register-sor.ts  POST /api/sor/route + execute tracking
├─ src/sor/dflow-pond-quote.ts        Pond fetch + overlay (internal)
├─ domain/dflow/order-quote-fetch.ts  shared Pond parse (not HTTP /order/quote)

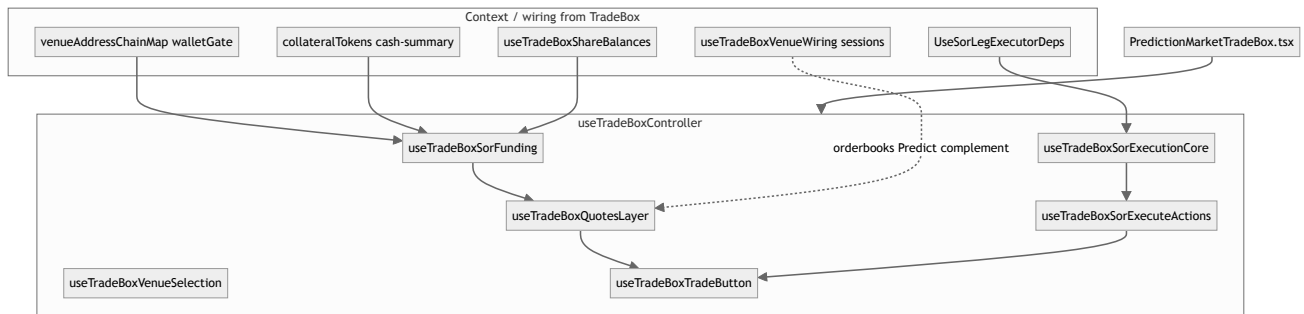
```

3. End-to-end trade path (the mental model)

Single path for quote and execute. No parallel DFlow quote hook.

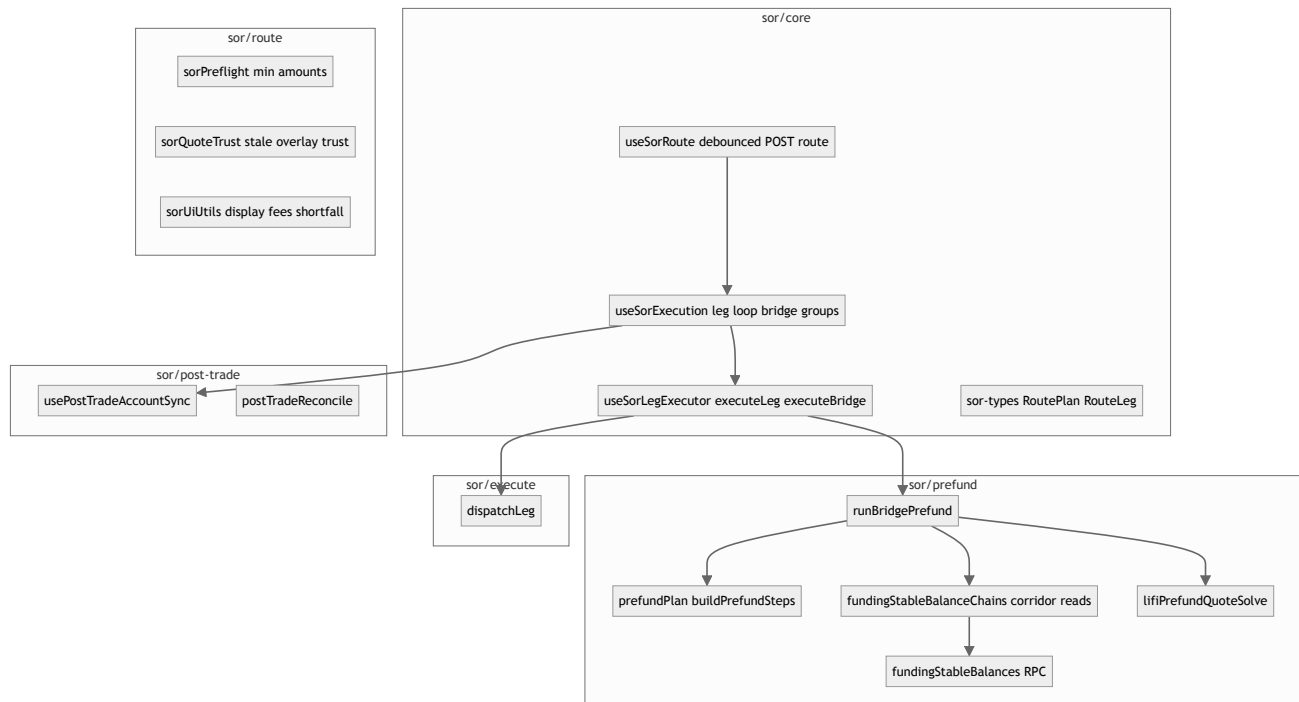


4. useTradeBoxController layers

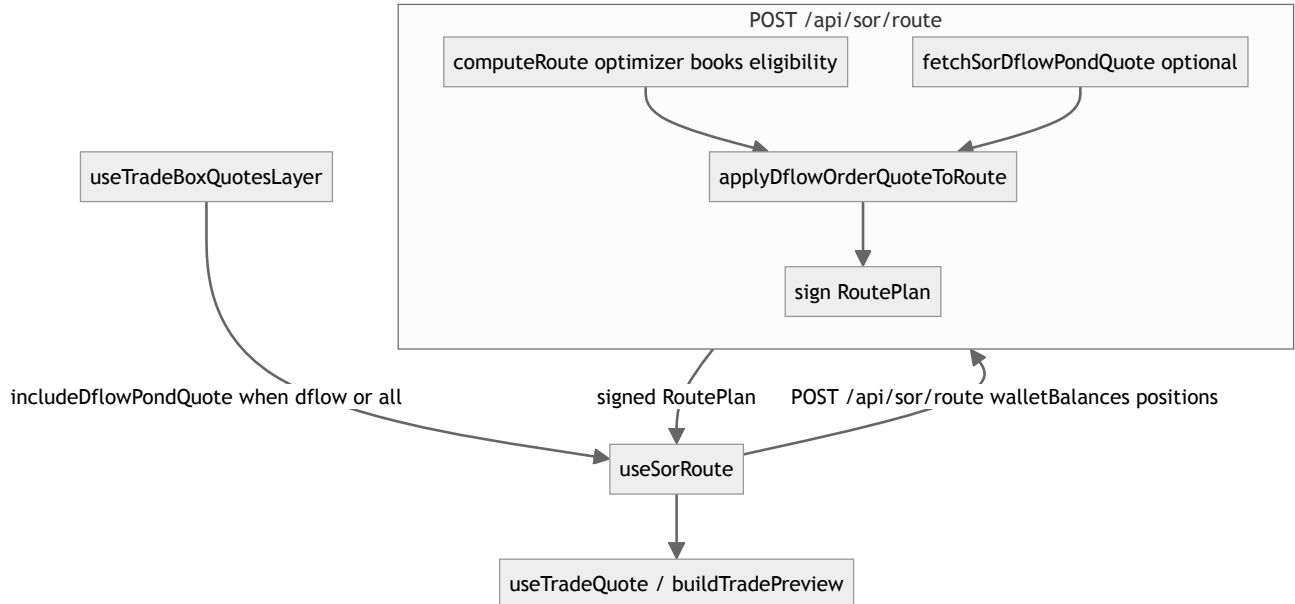


Hook	Responsibility
<code>useTradeBoxSorFunding</code>	<code>buildChainBalances</code> from VACM + cash → <code>walletBalances</code> on route request; venue position rows for sell
<code>useTradeBoxQuotesLayer</code>	Debounce <code>useSorRoute</code> (include <code>DflowPondQuote</code> for DFlow/all); merges into <code>useTradeQuote</code> for UI
<code>useTradeBoxSorExecutionCore</code>	<code>useSorLegExecutor</code> + <code>useSorExecution</code>
<code>useTradeBoxSorExecuteActions</code>	<code>handleSorExecute</code> , route expiry, post-trade baseline/reconcile
<code>useTradeBoxTradeButton</code>	<code>useButtonState</code> + <code>venueButtonState</code> — gates on VACM, cash, KYC, route errors

5. SOR client modules

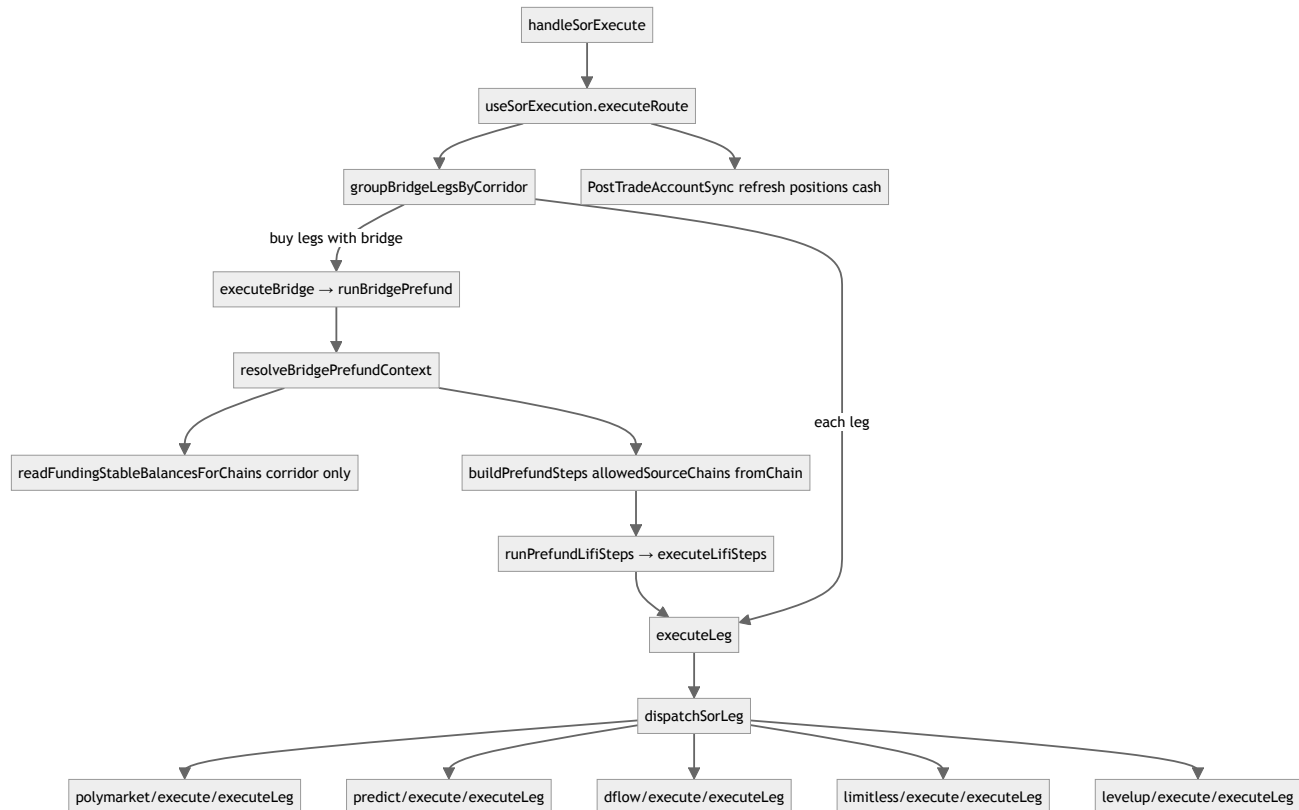


6. Unified quote (client + server)



Removed: `useDflowOrderQuote.ts` (client), `GET /api/dflow/order/quote` (server). DFlow execution still uses `GET /api/dflow/order` at leg time via `quoteSignAndSubmitDflowOrder.ts`.

7. Execute path



Prefund corridor (not “always Base”)

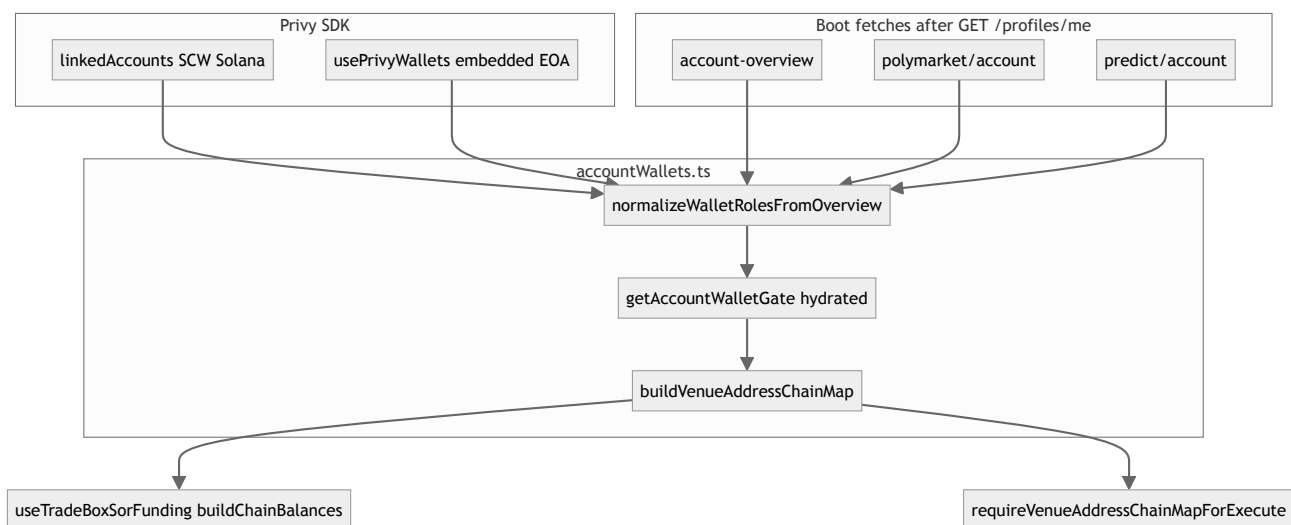
SOR planner sets `leg.bridge.fromChain` → `toChain`. Execute reads balances only for that corridor (`chainsForBridgeCorridor`). LiFi steps use `allowedSourceChains`: `[bridge.fromChain]` — one source chain per bridge group, chosen by the optimizer.

fromChain	LiFi fromAddress (VACM)	Notes
base	SCW or Limitless maker	Two Base pools; optional same-chain SCW→maker sweep before LiFi
polygon	Polymarket deposit safe	May use Poly relay for signing
bnb	Embedded EOA	USDT on BSC
solana	Solana embedded	Privy Solana sign

Cash already on the **destination** chain counts toward the prefund target first (`computePrefundBridgeShortfallUsdHuman`), reducing LiFi pull.

8. VACM hydration

Privy alone does not produce VACM. `AccountDataProvider` merges four slices, then `normalizeWalletRolesFromOverview` → `buildVenueAddressChainMap`.



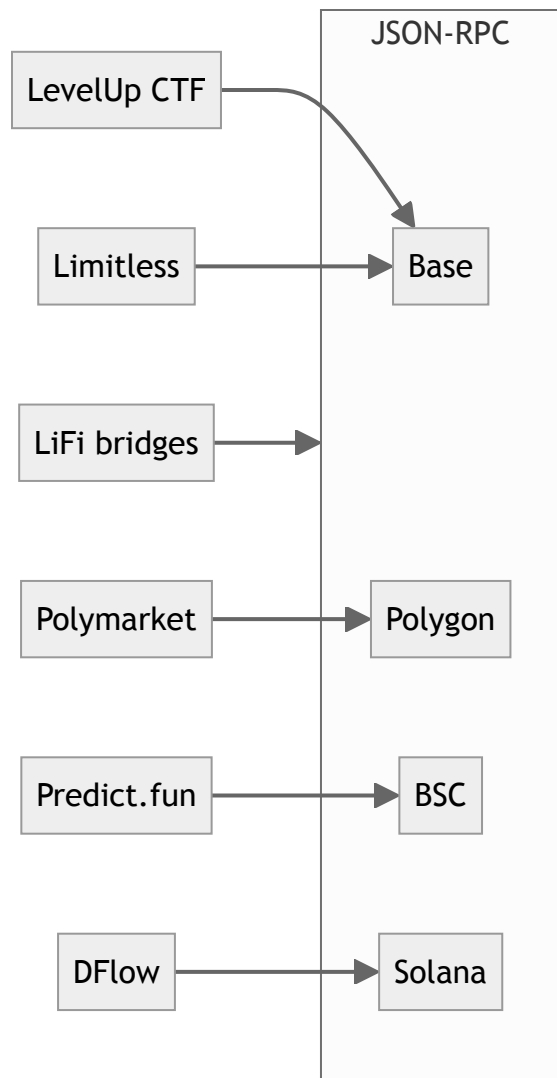
Role	Primary source	Fallback
baseSmartWallet	overview smart wallet row	Privy smart_wallet linked account
embeddedEoa	Privy embedded EOA	—
polymarketSafe	polymarket/account safeWalletAddress	overview polymarket fundingDestination
polygonSigner	polymarket/account signerAddress	embeddedEoa
predictMaker	predict/account makerAddress	overview → embeddedEoa
limitlessMakerBase	overview limitless fundingDestination	—
solanaAddress	Privy Solana wallet	overview / linkedAccounts

VACM → venue (signer vs collateral wallet)

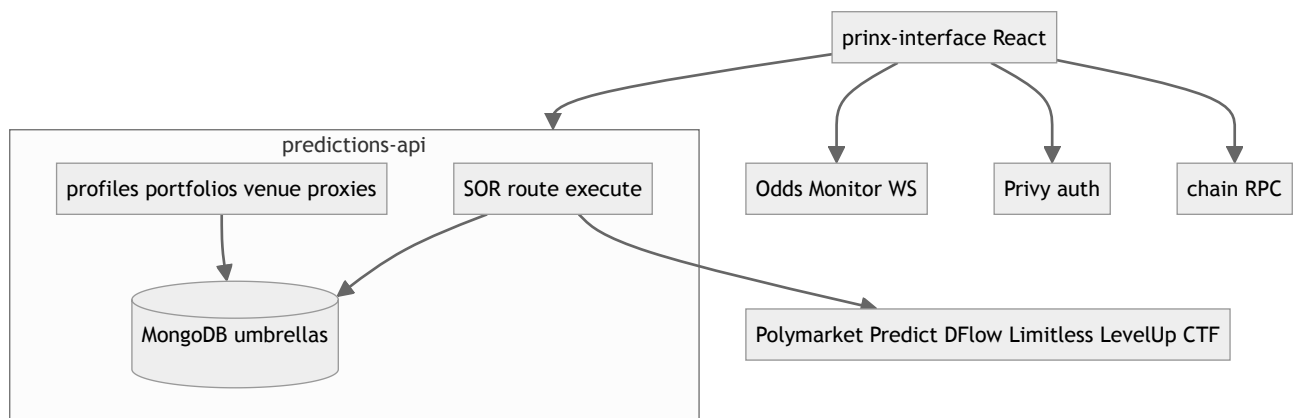
Venue	Chain	walletAddress	signerAddress
levelup	base	SCW	SCW
limitless	base	embedded EOA	embedded EOA
polymarket	polygon	deposit safe	polygon signer EOA
predictfun	bnb	predict maker	embedded EOA
dflow	solana	solana wallet	same

9. RPC networks vs wallets

JSON-RPC chains (Base, Polygon, BSC, Solana) are **transport**. VACM is **who signs and where collateral sits**. Limitless and LevelUp both use Base RPC but different wallets.



10. System context (brief)



Provider stack (mount order): Privy → PredictionData → OddsMonitor → Signer → AccountData → UserData → venue background activators → routes → StickyTradeAmount → TradeBox → SOR hooks.

11. Predict.fun single-market fix (refactor-2)

Predict REST orderbook is YES-native. Sell NO requires complement before sizing:

- `predict/book/predictSingleMarketBook.ts` — complement helpers
- `useTradeBoxVenueWiring.ts` — display book complement
- `usePredictTradingSession.ts` — `complementOrderbook` flag
- `useSorLegExecutor` / `predict executeLeg` — complement at execution from leg outcome + monitor

12. Known gaps (not done in refactor-2)

- **Wallet gate incomplete** — `isTradeFundingReady` / button state partial; some legacy `?? 0` paths may remain.
- **Predict success UX** — toast on HTTP 201 even if order never fills; no mandatory order-status poll yet.
- **LiFi stale quote** — Across deadline can expire between quote and SCW estimate (intermittent prefund revert); re-quote before send not implemented.
- **Limitless** — verify separately; was broken at some branch checkpoints.
- **predictions-api build** — unrelated TS error in venue-connection-manager at last check.

13. Verify commands

```
cd prinx-interface
yarn build:strict
npx vitest run src/trading/predict/predictSingleMarketBook.test.ts \
  src/trading/sor/__tests__/prefundPlan.test.ts \
  src/trading/sor/__tests__/fundingStableBalanceChains.test.ts

cd predictions-api
yarn run build
npx vitest run src/sor/__tests__/dflow-pond-quote.test.ts
```